

Declarative Kotlin

Alexander Kuklev^{1,2} a@kuklev.com

¹Radboud University Nijmegen, Software Science

²JetBrains Research

Declarative programming is an approach striving to enable describing what you want to get rather than how to get it. It facilitates concise, transparent, straight-to-the-point code and is an indispensable tool for tackling inherent complexity. Here, we outline how to introduce declarative programming capabilities into Literage Kotlin, a language with great support for declarative DSLs including an SQL-like reactive query language. Full-scale implementation of our proposals brings the full power of functional logic programming (roughly “Haskell + Prolog”), while already limited support extends the query language with an accessible form of Datalog.

1 Implicit definitions

Implicit definitions enable declarative programming whenever objectives can be described by conditions. They are ubiquitous in mathematical texts, so supporting the widest possible class of them is highly desirable for a language used in academia. We propose the following notation:

let $x\ y : \text{Float}$ $x + 2y = 5$ $x - y = 4$	let $\text{gcd} : \text{Int}$ $n \% \text{gcd} = 0$ $m \% \text{gcd} = 0$ maximizing { gcd }	try let $x\ ?t : \text{Float}$ $x = a + b \cdot t$ $x = c + d \cdot t$
---	--	---

A **let** block contains conditions imposed on the indeterminates declared in its header. Conditions must uniquely determine the values of the indeterminates except for so-called existential variables (marked like $?t$), which are scoped within the block and not exposed. A **let** block can only be compiled if there is an appropriate solver for conditions of the given form on indeterminates of given types. Solvers have to ensure the existence of a unique solution,¹ either at compile-time (**let** blocks) or at run-time (**try let** blocks). At present, we envision three specialized solvers:

- the $*$ -semiring linear equation solver,²
- the mixed integer and real linear arithmetic solver,³
- an SAT/SMT (boolean satisfiability/satisfiability modulo theories) solver.

2 Indeterminate functions

The expressivity of conditions, not only in **let** blocks but also in ordinary conditionals and filters, can be greatly improved with indeterminate functions. While ordinary functions compute *the* value for given arguments, indeterminate functions describe what’s *a* value for given arguments:

```
data class Person(val father : Person, val mother : Person)

def a Person.parent = anyOf(this.mother, this.father)           # Just 2 values
def an Person.ancestor = anyOf(this.parent, this.ancestor.parent) # Recursion!

persons ▶filter { it.ancestor = x }
```

We define what’s *a* .parent and *an* .ancestor of a person to **▶filter** those with *an* ancestor *x*. Such code is much leaner than any imperative alternative, leaving no place for a bug to hide.

A query $f : \text{query } (Xs) \rightarrow Y$ either computes a value and returns it together with a deferred query for alternative values, or recognizes lack of value(s). A general indeterminate computable function $f : \text{quest } (Xs) \rightarrow Y$ is a query that may fail to return at all. In both cases, *f* is not allowed to have side effects and can only be invoked inside other queries/indeterminate functions, or converted into potentially infinite streams $\text{all}\{f\} : (Xs) \rightarrow \text{Sequence}\langle Y \rangle$.

¹Take $a = c$, $b = d = 0$ in the rightmost example. Its solution $x = c$ qualifies as unique because t is existential.

²<https://r6.ca/blog/20110808T035622Z.html>

³https://doi.org/10.1007/978-3-030-55754-6_14

3 Indeterminate implicit definitions

Verse calculus,⁴ a novel approach based on introspectable indeterminate functions, provides unrestricted `let` blocks with arbitrary conditions expressible in terms of computable functions.

This extends applications of `let` blocks far beyond academic purposes, into the realm of complex real-world applications, where the inherent complexity of problems and systems leads to intricate, fragile, and error-prone code, unless systematically managed declaratively.

Inside indeterminate functions, existence and uniqueness restrictions for `let` blocks can be lifted, and solvers are merely required to produce an effectively exhaustive⁵ stream of solutions⁶ x_i . Given a computable $f : (\text{Int}) \rightarrow \text{Int}$, indeterminate functions can employ `let` blocks like this:

```
let x : Int
  f(x) = c
```

It is valid because one can brute-force a stream of solutions by successively applying f to all possible integers $(0, \pm 1, \pm 2, \dots)$ until the result turns out to be equal to the constant c . One could have used multiple variables and conditions, an indeterminate f , any predicate instead of $(= c)$ as long as it is expressible by a computable function (i.e. semi-decidable), and any type X instead of `Int` as long as it admits an effectively exhaustive sequence $x_i : X$ (i.e. “surveyable”).⁷

The prime purpose of declarative programming is to provide self-evidently correct reference definitions, regardless of their efficiency, but brute-forcing is far too inefficient to ever be feasible in practice. Fortunately, Verse calculus offers an ingenious approach that turns out to be reasonably effective. In the case of queries applied to finitary data such as databases and data streams, there is a whole arsenal of highly efficient Datalog query optimizations in addition.

4 Conclusion and outlook

By leveraging declarative programming, we can substantially improve the expressivity of Kotlin and extend its capabilities to a whole new level. These extensions make it a superior choice not only for literate programming and illustrating ideas in teaching and research, but also for a variety of real-world applications where the taming of inherent complexity is urgently needed.

Indeterminate functions in queries virtually eliminate the need for subqueries and dramatically reduce the code in size and complexity, making it easier to understand and less error-prone. Functional logic programming provides a tractable approach to dealing with vast combinatorial complexity, e.g. in conflict resolution for interacting software systems and constraint satisfaction.

When it comes to academic and educational use, implicit definitions help to keep code as close to the text as possible, or to provide straightforward equivalents for intricate algorithms so as to elucidate all subtleties and catch all the bugs while establishing the equivalence.⁸

While being remarkably useful, both pure functional programming (à la Haskell) and logic programming (à la Prolog) have a reputation for being arcane academic gimmicks. The recently developed Verse calculus has finally made it possible to combine them into functional logic programming, which is generally assumed to be even less capable of gaining broad adoption in the foreseeable future. Our approach aims to wield the full power of functional logic programming without expecting the users to have any understanding of the underlying calculus. To this end, we discreetly introduce functional logic programming into a mainstream programming language using merely two non-intrusive, reader-friendly, and self-explanatory constructs.

⁴<https://simon.peytonjones.org/assets/pdfs/verse-icfp23.pdf>

⁵For every semi-decidable predicate P that holds for some solution, there must be at least one x_n satisfying P . This condition reduces to exhaustiveness for enumerable types and types with semi-decidable equality.

⁶The type `quest T` of indeterminate T -valued computations is an instance of the $?T$ modality in linear logic.

⁷A condition satisfied by all enumerable and many relevant non-enumerable types like $\mathbb{R}$.

⁸For instance, `gcd` as defined in §1 is an equivalent for the non-trivial Euclidean algorithm (except $n = m = 0$):

```
def gcd(n m : Nat) = if(m = 0) n else gcd(m, n % m)
```